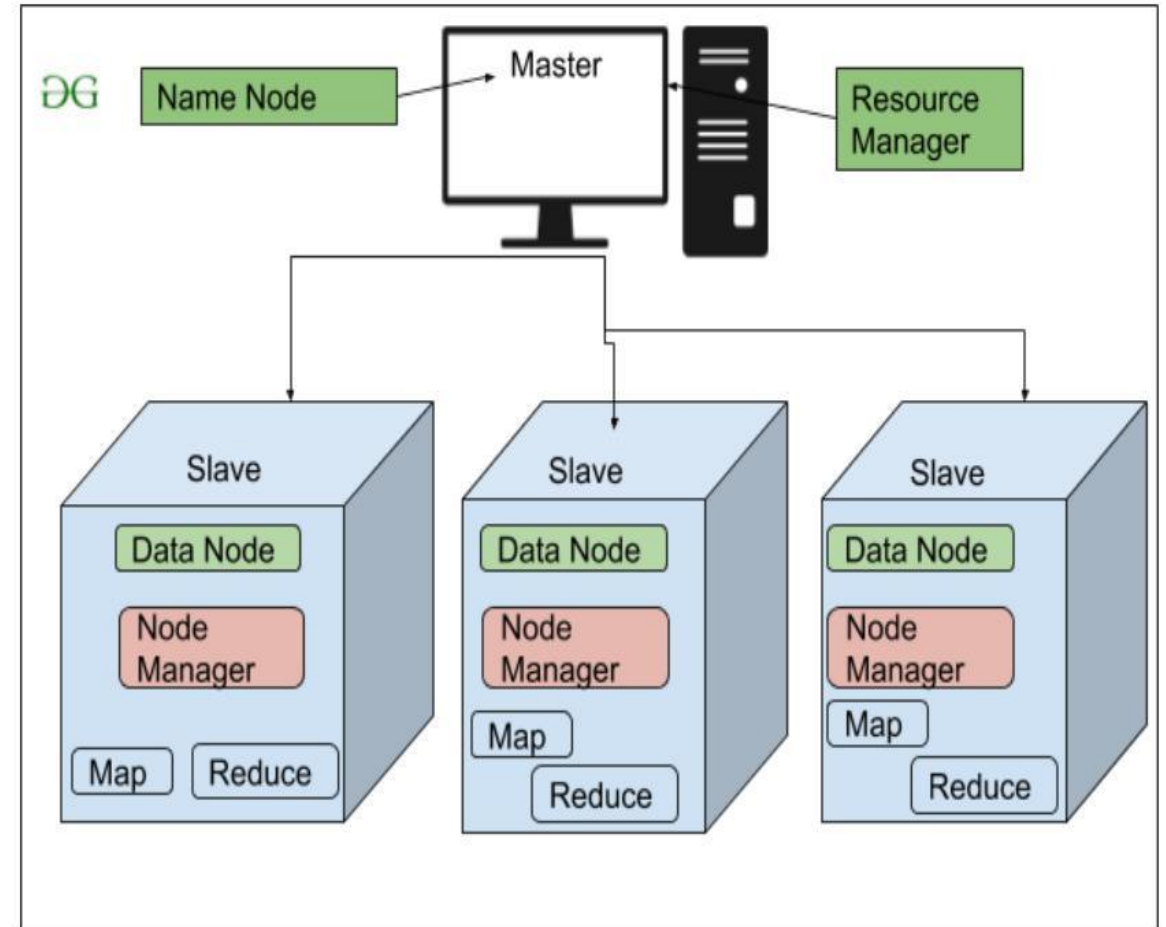
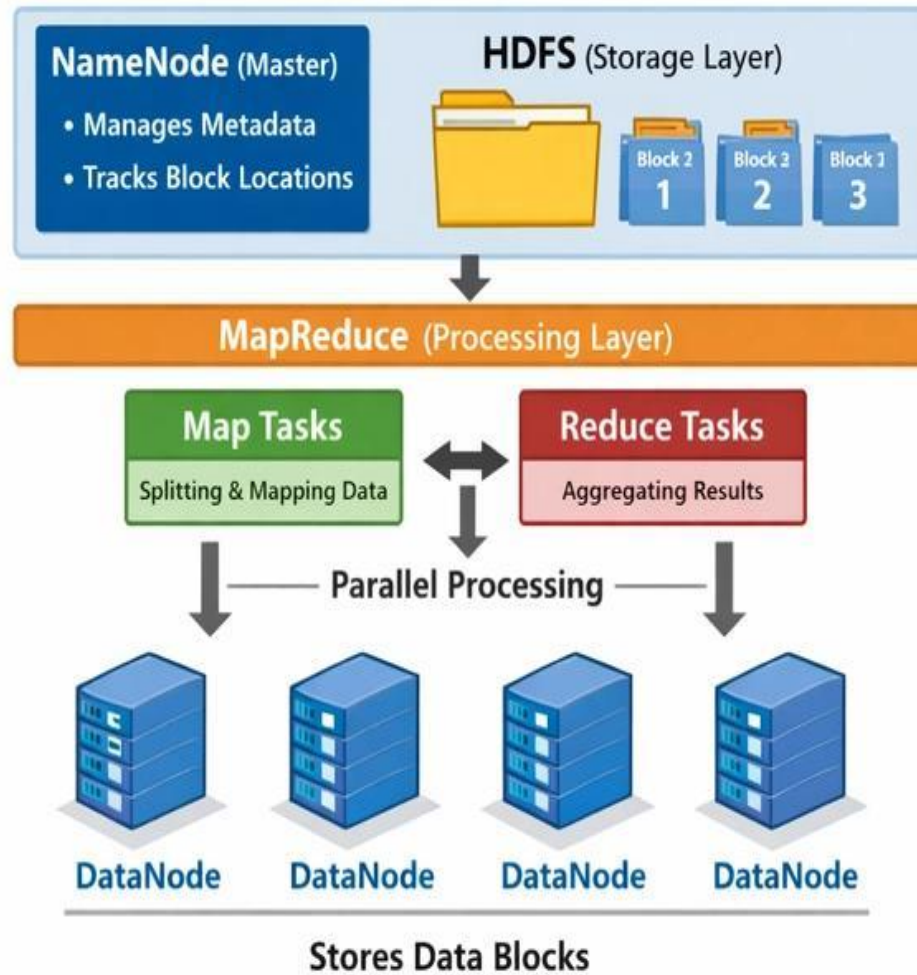


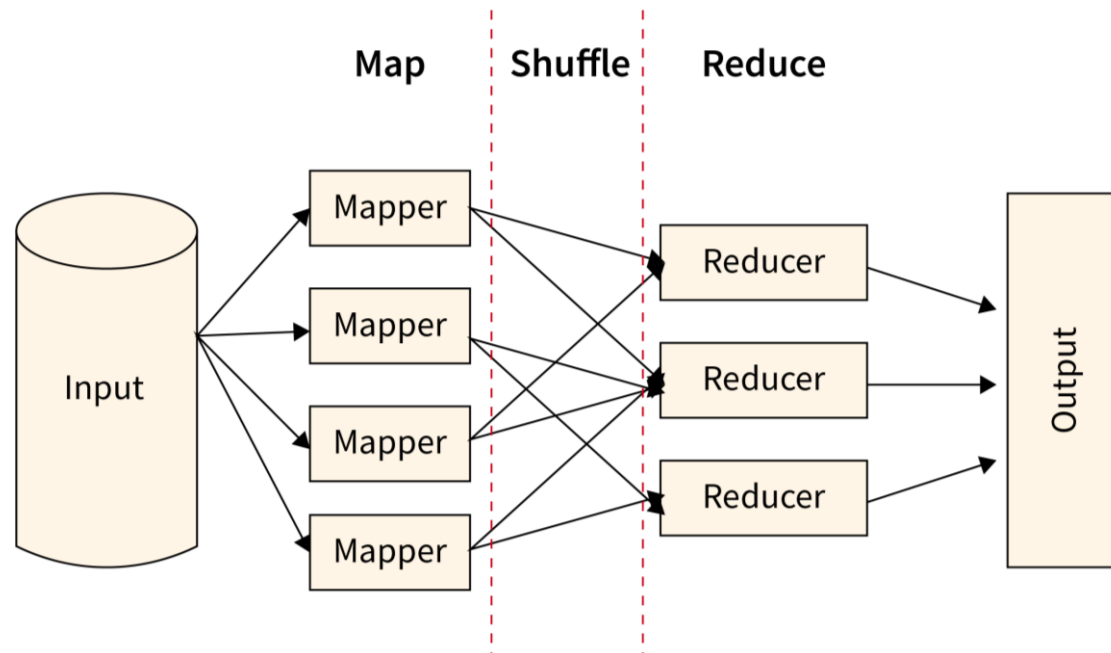
MAPReduce

Hadoop Architecture: HDFS Storage and MapReduce Processing

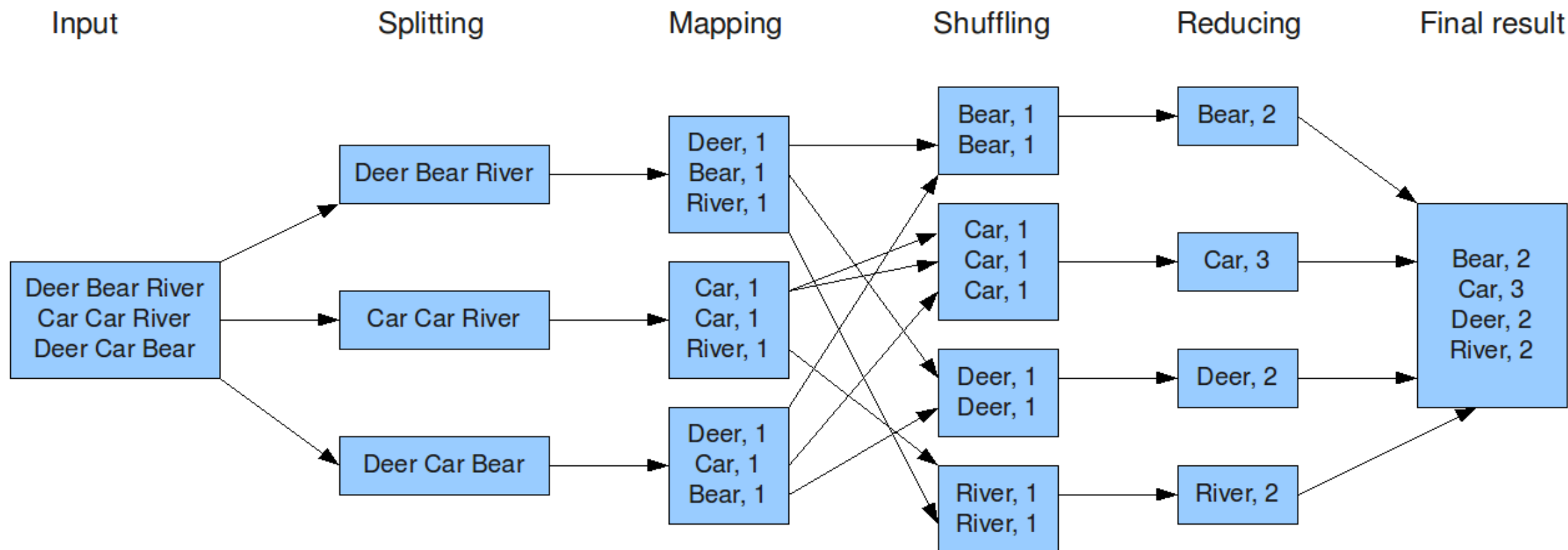


Introduction

- MapReduce is a programming model and processing technique used to run massive data tasks across many computers at the same time. It splits work into a **Map** step to filter and sort data into key-value pairs, and a **Reduce** step to combine those values into a final summary.



The overall MapReduce word count process



MAP Stage

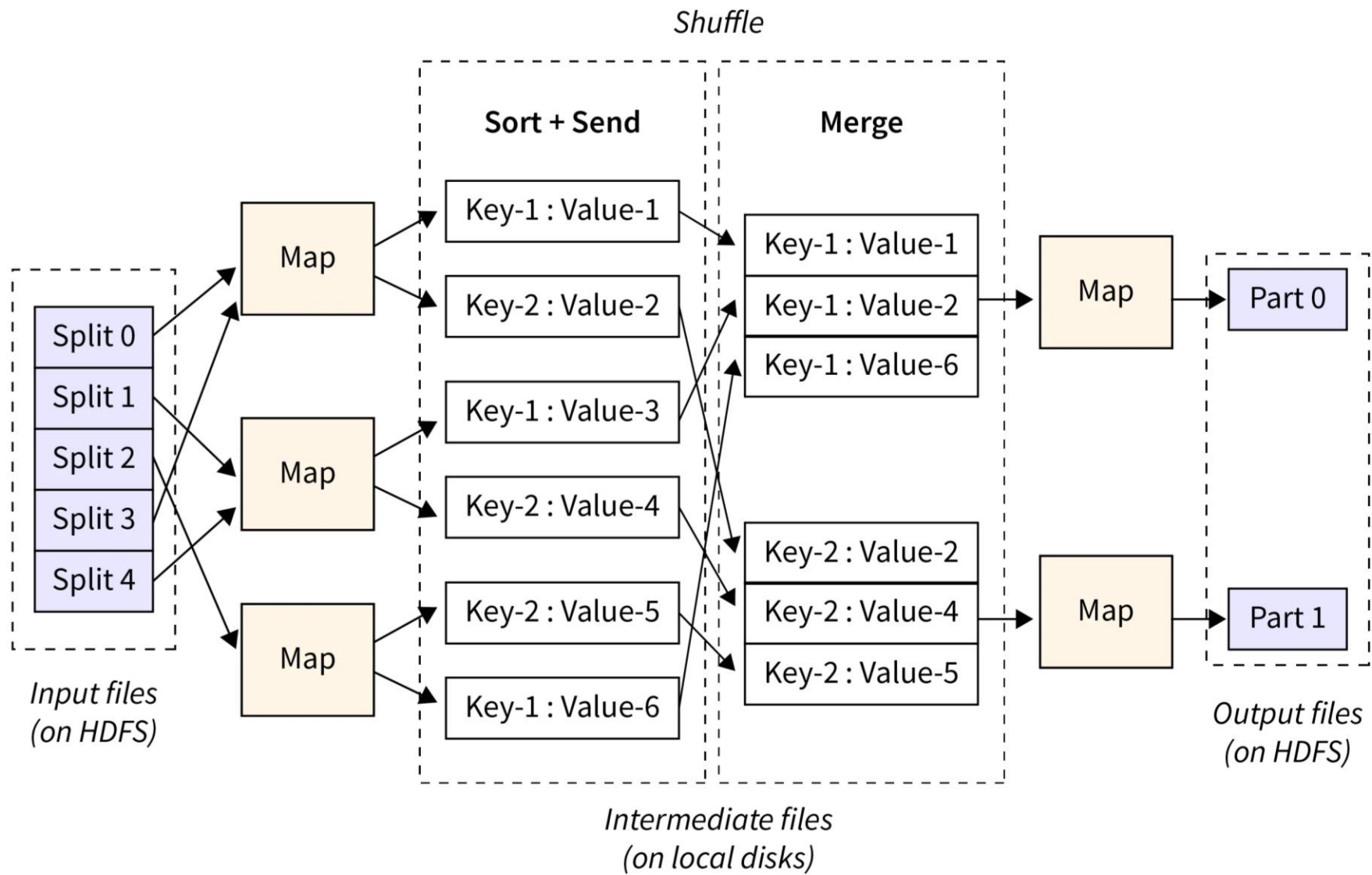
- Input data is divided into smaller chunks or blocks.
- Several worker nodes work in parallel to process each chunk independently.
- A “Map” function is applied to each data chunk, generating intermediate key-value pairs.
- The Map function's goal is to extract relevant information from the input data and prepare it for further processing.

Reduce Stage

- After the Map stage, the intermediate key-value pairs are grouped by key.
- The grouped key-value pairs are then shuffled and sorted based on their keys.
- The purpose of the shuffle and sort phase is to bring together all the intermediate values associated with the same key and make them available to the corresponding Reduce function.
- Once the shuffling and sorting are complete, a "Reduce" function is applied to perform aggregation, analysis, or other computations on the grouped data.
- The output is a set of final key-value pairs from the computation.

MAPReduce Framework Operations

- **Map:** each worker node applies the map function to the local data, and writes the output to a temporary storage. A master node ensures that only one copy of the redundant input data is processed.
- **Shuffle:** worker nodes redistribute data based on the output keys (produced by the map function), such that all data belonging to one key is located on the same worker node.
- **Reduce:** worker nodes now process each group of output data, per key, in parallel.

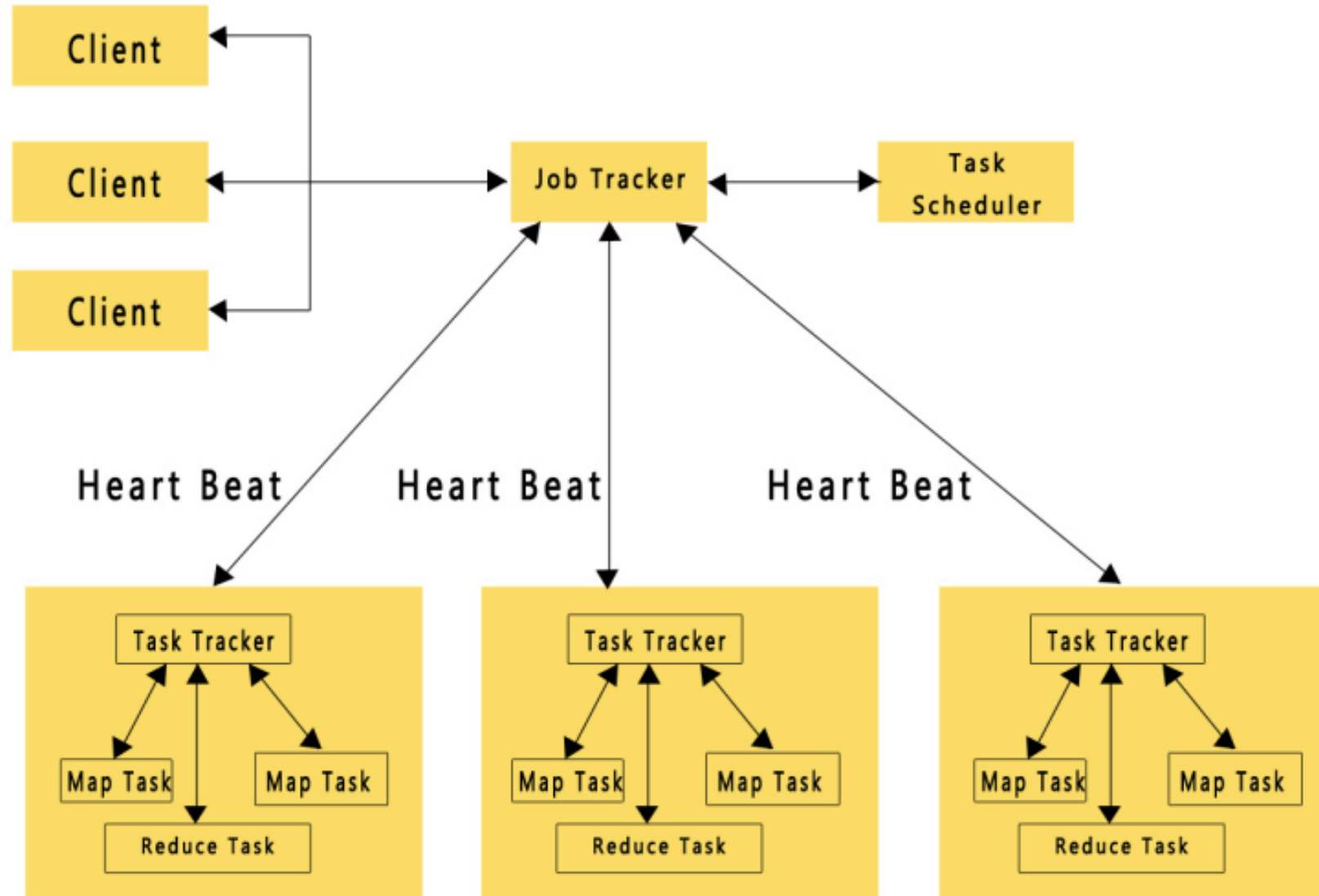


MapReduce Data Processing Flow

- **Input Splitting:** First, the input data stored in HDFS is divided into multiple smaller parts called input splits. These splits allow the data to be processed in parallel.
- **Map Phase:** Each split is assigned to a Map task. The mapper processes its input and generates intermediate key–value pairs, such as (Key-1, Value-1) and (Key-2, Value-2).
- **Sort and Send:** The intermediate key–value pairs are then sorted based on their keys. The data is partitioned and prepared to be sent to the appropriate reducers.
- **Shuffle Phase:** During the shuffle phase, all values belonging to the same key are transferred and grouped together. For example, all Key-1 values are collected together, and all Key-2 values are collected together.
- **Merge Phase:** The reducer merges the sorted data received from different mappers. Thus, we obtain groups such as:
Key-1 → Value-1, Value-2, Value-6
Key-2 → Value-2, Value-4, Value-5
- **Reduce/Final Processing:** The grouped key–value pairs are processed to produce the final result. The output is written as separate files, such as Part 0 and Part 1, back into HDFS.

Input Data → Splitting → Map → Sort → Shuffle → Merge → Reduce → Output

MAPReduce Architecture



MAPReduce Architecture

- 1. Client:** The client submits a MapReduce job to the JobTracker. Multiple clients can submit jobs to the Hadoop cluster.
- 2. JobTracker (Master Node):** The JobTracker is the central master component of MapReduce. Its main responsibilities are:
 - Receiving jobs from clients.
 - Dividing the job into multiple Map and Reduce tasks.
 - Scheduling these tasks on available worker nodes.
 - Monitoring the execution of tasks.
 - Handling task failures and reassigning failed tasks.

MAPReduce Architecture (Contd...)

3. Task Scheduler: The Task Scheduler works with the JobTracker to decide which task should run on which worker node. It attempts to efficiently allocate resources and schedule Map and Reduce tasks.

4. TaskTrackers (Worker Nodes): Each worker machine runs a **TaskTracker**. Main functions are:

- Receives tasks from the JobTracker.
- Executes **Map tasks** and **Reduce tasks**.
- Monitors their execution.
- Reports its status back to the JobTracker.

MAPReduce Architecture (Contd...)

5. Map and Reduce Tasks: Inside each TaskTracker:

- **Map Tasks** process the input data and generate intermediate key–value pairs.
- **Reduce Tasks** receive the intermediate results, process them, and generate the final output.

6. Heartbeat Mechanism: The arrows labeled Heartbeat represent regular communication between the TaskTrackers and the JobTracker.

- Through these heartbeat messages:
- TaskTrackers report their availability and status.
- The JobTracker monitors whether worker nodes are alive.
- Information about completed or failed tasks is communicated.
- New tasks can be assigned to available TaskTrackers.